

WEB

Résumé de chaque méthode

- **app.get()** : Utilisé pour **obtenir des informations** quand quelqu'un visite une page de ton site.
- **app.post()** : Utilisé pour **envoyer des informations** à ton serveur depuis le client.
- **app.listen()** : Démarre ton serveur pour qu'il puisse écouter les requêtes entrantes.

6. Tester ton application

Quand tu lances ton serveur avec la commande `node app.js` et que tu visites `http://localhost:3000/` dans ton navigateur, tu verras le message que tu as programmé dans le `app.get()`. Pour tester la méthode POST, tu utilises la commande `curl` pour envoyer des données à ton serveur, et tu verras ces données s'afficher dans la console.

1. Qu'est-ce qu'un Middleware ?

Le middleware sert à intercepter les requêtes et faire des choses avant de répondre. Dans cet exemple, le middleware est utilisé pour enregistrer chaque demande dans la console. C'est super utile pour suivre l'activité de ton application et voir les requêtes qui arrivent !

Imaginons que tu vas à un guichet. Le **middleware** serait comme une personne qui te demande ton ticket avant que tu puisses rencontrer le guichetier. Une fois que la personne a validé ton ticket, elle te laisse passer pour que tu puisses parler au guichetier (c'est ton serveur).

Qu'est-ce qu'un middleware tiers ?

Un **middleware tiers** est comme un outil ou une fonction déjà écrite par quelqu'un d'autre, que tu peux ajouter dans ton application pour faire des tâches spécifiques sans avoir à tout réécrire toi-même.

Exemple : Morgan (middleware tiers pour l'enregistrement des requêtes)

Morgan est un **middleware tiers** qui sert à enregistrer automatiquement chaque requête HTTP (comme GET ou POST) dans la console. C'est comme si tu avais un assistant qui note tout ce qui se passe sur ton site.

Pourquoi c'est utile ?

Morgan te permet de **voir** dans la console ce qui se passe dans ton application, et ça t'aide à savoir :

- Qui accède à ton site.
- Combien de temps ça prend pour répondre.
- Si des erreurs se produisent.